Document number: P0091R4
Revision of P0091R3
Date: 2017-02-06
Reply-To:
Mike Spertus, Symantec (mike_spertus@symantec.com)
Faisal Vali (faisalv@yahoo.com)
Richard Smith (richard@metafoo.co.uk)
Audience: Evolution Working Group

Template argument deduction for class templates (Rev. 7)

This paper discusses possible extensions to template argument deduction for class templates as described in P0091R3. Please see that paper for background and context.

Deleted deduction guides

When C++ automatically generates functions or methods, it generally allows the programmer to suppress the generation of the function/method through the use of `= delete;`. E.g.

```
struct A {
  A(const A &) = delete;
};

template<typename T>
int f(T t) { return 3; }
template<>
int f<double>(double) = delete;
```

It seems very odd that ordinary template functions can be deleted but deduction guides cannot. Indeed, we contend that supporting `= delete` for deduction guides not only increases consistency but has worthwhile use cases as well.

Consider the following example from "P0433R1: Toward a resolution of US7 and US14: Integrating template deduction for class templates into the standard library."

```
int iarr[] = {1, 2, 3};
int *ip = iarr;
valarray va(ip, 3); // Deduces valarray<int>
```

The point is that the `valarray<T>::valarray(const T *, size_t)` constructor is a better match than the `valarray<T>::valarray(const T &, size_t)` constructor, so `valarray<int>` is preferred over `valarray<int *>`. Given the typical usage of `valarray`, we believe that this is a reasonable default, and that is what we recommend in P0433R1.

However, suppose we had wanted this deduction to be ambiguous for `valarray` or a similar class. The natural way to do this would be to delete the implicit deduction guide that takes a pointer.

```
template<T> valarray(const T *, size_t) -> valarray<T> = delete;
```

Wording

Modify §8.4.3 [dcl.fct.def.delete] as follows:

> A function definition of the form:
>
> *attribute-specifier-seq*$_{opt}$ *decl-specifier-seq*$_{opt}$ *declarator virt-specifier-seq*$_{opt}$ `= delete ;`
>
> is called a *deleted definition*. A *deduction-guide* (14.9) with a deleted body is also a *deleted definition*

Modify §14.9 [temp.deduct.guide] as follows.

> *deduction-guide:*
> `explicit`$_{opt}$ *template-name* `(` *parameter-declaration-clause* `) ->` *simple-template-id* *function-body*$_{opt}$ `;`

Add the following sentence to the end of 14.9p3:

> A *deduction-guide* shall be declared in the same scope as the corresponding class template. If the *function-body* is not `= delete` or `= default`, the program is ill-formed.

Copy/move constructor suppression

Consider the following example from P0433R1.

```
optional o(3);  // Deduces optional<int>
optional o2 = o;  // Deduces optional<int>
```

While this seems natural for user code, it is awkward for writers of template libraries

```
template<typename T> void foo(T t)
{
  optional o = t; // T=optional<int> => o is optional<int>, which may not be desired in generic code
  optional<T> o2 = t; // o2 is optional<optional<int>>
}
```

While library writers can get around this by using explicit arguments as above, we could consider the possible extension that constructing with a single-element braced initializer would be defined to make the deduction guides implicitly generated from the copy and move constructors the least viable overload. viable constructors.

```
template<typename T> void foo(T t)
{
  optional o = {t}; // o is now always optional<T>
}
```

There are a number of benefits to this approach. It would make it easier for library writers to write generic code without worrying whether they will invoke a copy constructor. Furthermore, "`optional o = {t};`" is not a natural way to invoke a copy constructor anyway. Even better, since template constructor deduction is new, there are no backwards-compatibility issues to worry about.

The main concern is that this rule does not overlap with natural code for invoking a copy constructor. While we believe the `= {t};` approach avoids excessive overlap. If direct initialization is needed, code such as

```
optional o{t}; // Want to leave as copy constructor
```

is not appealing as it is commonly used for copy construction (also, cf. CWG 1467). However, the following syntactically correct but less traditionally natural code can help to avoid the overlap.

```
optional o({t}); // Seems OK to prefer optional<T>::optional(T const &) constructor
```

The other main objection to copy/move constructor suppression is that it is admittedly a hack to get around a problem that could also be avoided by using the C++14 approach of explicitly specifying `optional<T>`. Still, worthwhile template programming notations are sometimes considerably more convoluted than this. so we will leave to EWG to decide.